

Programs

1. **Build a basic server using Node.js and Express. Create routes to handle HTTP requests like GET and POST and respond with simple JSON data.**
2. **Extend the previous Node.js server by integrating a database (SQLite). Implement endpoints to perform CRUD operations on a dataset.**
3. **Design and implement a RESTful API using Node.js, Express, and a database. Define endpoints for managing resources (CRUD operations).**
4. **Create a library of reusable React components including buttons, cards, and modals and use them in a sample React application.**
5. **Write a program to create a simple calculator application using React**
6. **Create a simple Login form using React JS.**
7. **Build a real-time chat application using React for the front end and Node.js with Socket.IO for the back end.**

- 8. Develop a Django app that displays current date and time in server.**

- 9. Develop a simple Django app that displays an unordered list of fruits and ordered list of selected students for an event.**

- 10. Develop a Django app that performs student registration on a course with models, enrollment as many-to-many fields, and displays registered students.**

Program 1 : Build a basic server using Node.js and Express. Create routes to handle HTTP requests like GET and POST and respond with simple JSON data.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create a Project Folder

Create a new folder named Program1 and open it in VS Code.

Step 3 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 4 : Initialize a Node.js Project

```
npm init -y
```

This creates a default package.json file.

Step 5 : Install Express Package

```
npm install express
```

This installs the Express framework

Step 6 : Create app.js File

Project Structure

Program1/

```
|  
|— node_modules/  
|— package.json  
|— app.js
```

Step 7 : Write the Following Code in app.js

```
// Import Express framework  
const express = require("express");  
  
// Create Express application  
const app = express();  
  
const PORT = process.env.PORT || 3000;  
  
// Middleware setup  
app.use(express.json());  
app.use(express.static("public"));  
  
// Sample JSON data  
const data = [  
  { id: 1, name: "Abhishek" },  
  { id: 2, name: "Bhuvan" },  
  { id: 3, name: "Chetan" },  
  { id: 4, name: "Deksha" }  
];  
  
// Home route  
app.get("/", (req, res) => {  
  res.send("Home");  
});  
  
// GET route to fetch all data  
app.get("/api/data", (req, res) => {
```

```
    res.json(data);
  });

  // POST route to add new data
  app.post("/api/data", (req, res) => {

    const nItem = req.body;

    data.push(nItem);

    res.json(data);
  });

  // Start the server
  app.listen(PORT, (err) => {

    if (err) console.log(err);

    console.log("Server running on port:", PORT);
  });
});
```

Step 8 : Run the Application

node app.js

You should see:

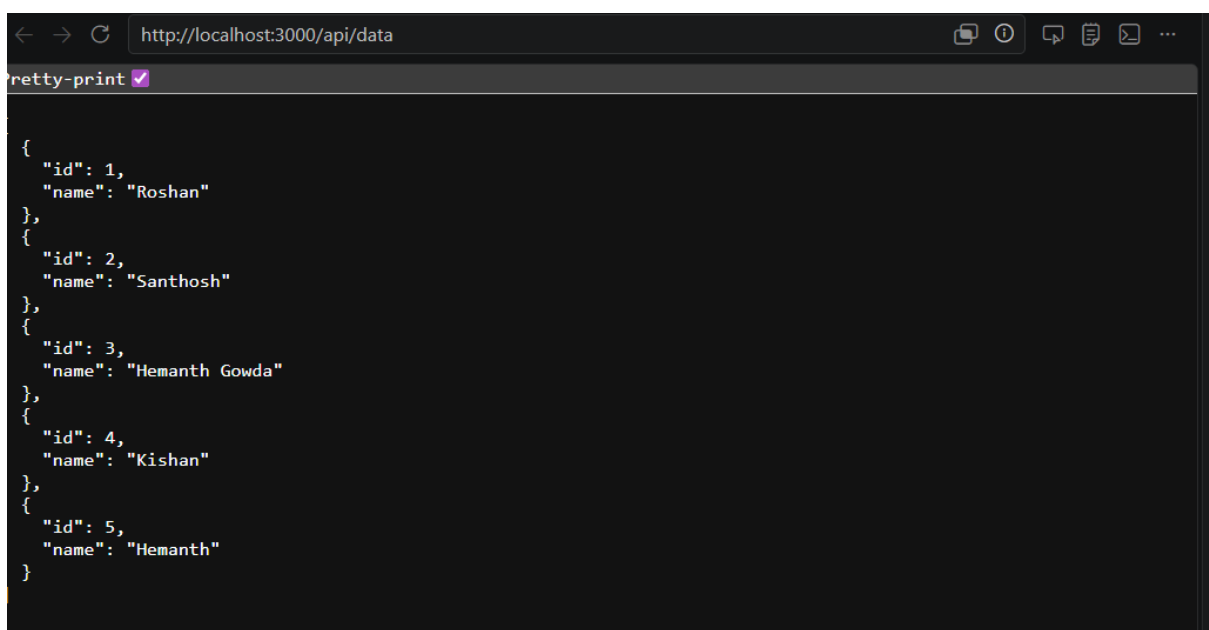
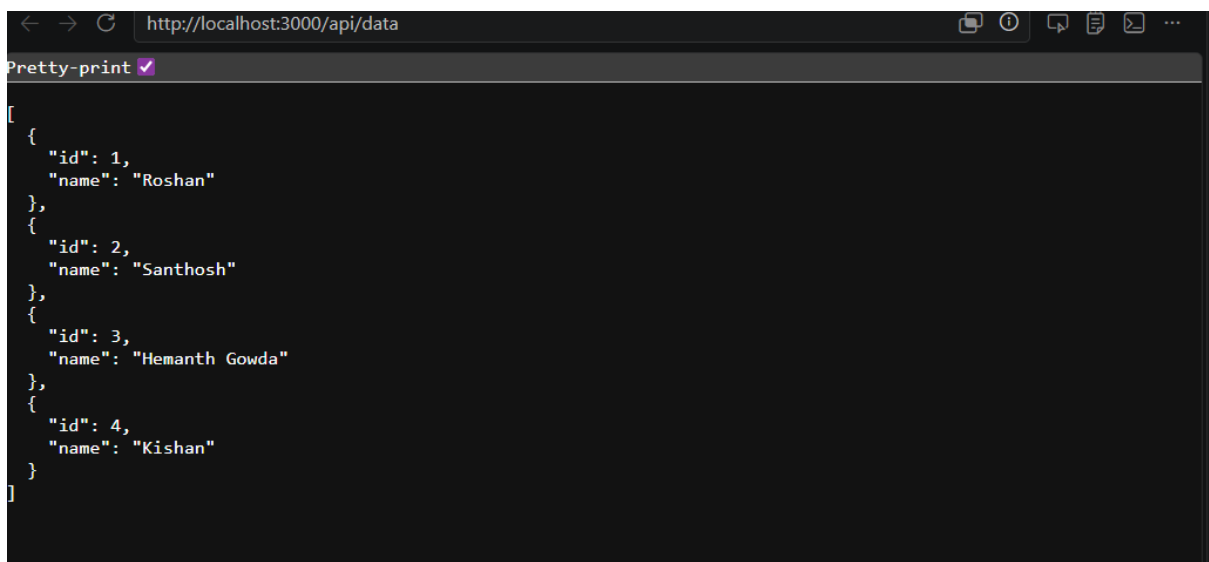
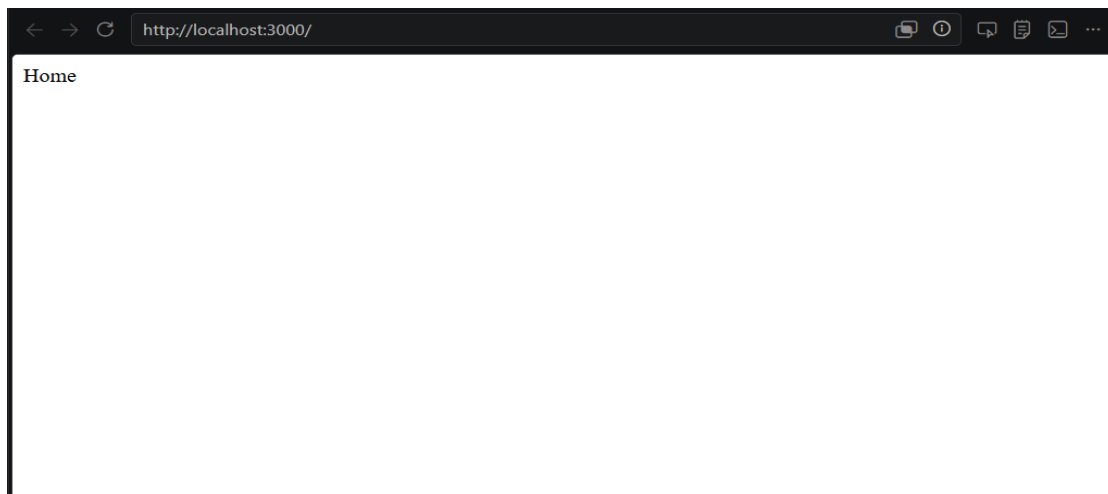
Server running on port: 3000

Step 9 : Open Browser and Test GET Route

http://localhost:3000/api/data

You will see JSON data in the browser.

Output



2. Extend the previous Node.js server by integrating a database (e.g., SQLite or MongoDB). Implement endpoints to perform CRUD operations on a dataset.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create a Project Folder

Create a new folder named Program2 and open it in VS Code.

Step 3 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 4 : Initialize a Node.js Project

```
npm init -y
```

This creates a default package.json file.

Step 5 : Install Required Packages

```
npm install express mongoose
```

This installs Express and MongoDB packages.

Step 6 : Create app.js File

Project Structure

Program2/

```
|— node_modules/  
|— package.json  
|— app.js
```

Step 7 : Write the Following Code in app.js

```
const express = require("express");  
const mongoose = require("mongoose");  
  
// Create Express application  
const app = express();  
  
const PORT = 3000;  
  
// Middleware setup  
app.use(express.json());  
  
// MongoDB connection  
mongoose.connect("mongodb://127.0.0.1:27017/studentDb")  
.then(() => console.log("MongoDB connected!"));  
  
// Student schema  
const studentSchema = new mongoose.Schema({  
  id: Number,  
  name: String,  
});  
  
// Create model  
const Student = mongoose.model("Student", studentSchema);  
  
// GET all data
```



```
app.get("/data", async (req, res) => {

  const data = await Student.find();

  res.json(data);

});

// GET data by ID
app.get("/data/:id", async (req, res) => {

  const data = await Student.findOne({
    id: Number(req.params.id)
  });

  res.json(data);

});

// POST new data
app.post("/data", async (req, res) => {

  const newData = new Student(req.body);

  const saved = await newData.save();

  res.json(saved);

});

// UPDATE existing data
app.put("/data/:id", async (req, res) => {

  const updated = await Student.findOneAndUpdate(

    { id: Number(req.params.id) },

    req.body,

    { new: true }
```

```
);

res.json(updated);

});

// DELETE data
app.delete("/data/:id", async (req, res) => {

  const deleted = await Student.findOneAndDelete({

    id: Number(req.params.id),

  });

  res.json(deleted);

});

// Start server
app.listen(PORT, () => {

  console.log(`Server running on port ${PORT}`);

});
```

Step 8 : Run the Application

node app.js

You should see:

Server running on port 3000

MongoDB connected!

Step 9 : Open Browser and Test API Routes

<http://localhost:3000/data>

You will see MongoDB data in JSON format.

Output

The image shows two screenshots. The top screenshot is of the MongoDB Compass interface. It displays the 'students' collection in the 'studentDB' database. The 'Documents' tab is active, showing three documents in a list view. Each document has an '_id' (ObjectId), a 'name', and an 'age'. The bottom screenshot is of a web browser at the URL 'http://localhost:3000/api/data'. It shows the output of an API call, which is a JSON array of five objects. Each object has an 'id' and a 'name'.

MongoDB Compass Data:

_id	name	age
ObjectId('69dfc91ef8e8a92a27371dee')	Hemanth	21
ObjectId('69dfcc9185fa38496c192b99')	santhosh	21
ObjectId('69dfccac4eba359386c8b92f')	santhosh	21

Web Browser Output (JSON):

```
{
  "id": 1,
  "name": "Roshan"
},
{
  "id": 2,
  "name": "Santhosh"
},
{
  "id": 3,
  "name": "Hemanth Gowda"
},
{
  "id": 4,
  "name": "Kishan"
},
{
  "id": 5,
  "name": "Hemanth"
}
```

Program 3 : Design and implement a RESTful API using Node.js, Express, and a database of your choice. Define endpoints for managing resources, such as creating, reading, updating, and deleting data.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create a Project Folder

Create a new folder named Program3 and open it in VS Code.

Step 3 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 4 : Initialize a Node.js Project

```
npm init -y
```

This creates a default package.json file.

Step 5 : Install Required Packages

```
npm install express mongoose
```

This installs Express and MongoDB packages.

Step 6 : Create app.js File

Project Structure

Program3/

```
|  
|— node_modules/  
|— package.json  
|— app.js
```

Step 7 : Write the Following Code in app.js

```
const express = require("express");  
const mongoose = require("mongoose");  
  
// Create Express application  
const app = express();  
  
const PORT = 3000;  
  
// Middleware setup  
app.use(express.json());  
  
// MongoDB connection  
mongoose.connect("mongodb://127.0.0.1:27017/studentDb")  
  .then(() => console.log("MongoDB connected!"))  
  .catch(err => console.log(err));  
  
// Student schema  
const studentSchema = new mongoose.Schema({  
  id: Number,  
  name: String,
```

```
});

// Create model
const Student = mongoose.model("Student", studentSchema);

// Home route
app.get("/", (req, res) => {

    res.send("Student API is running...");

});

// CREATE operation
app.post("/data", async (req, res) => {

    try {

        const newStudent = new Student(req.body);

        const saved = await newStudent.save();

        res.json(saved);

    }

    catch (err) {

        res.status(500).send("Error");

    }

});

// READ ALL operation
app.get("/data", async (req, res) => {

    try {

        const students = await Student.find();
```

```
        res.json(students);
    }

    catch (err) {

        res.status(500).send("Error");

    }

});

// READ ONE operation
app.get("/data/:id", async (req, res) => {

    try {

        const student = await Student.findOne({

            id: Number(req.params.id)

        });

        res.json(student);

    }

    catch (err) {

        res.status(500).send("Error");

    }

});

// UPDATE operation
app.put("/data/:id", async (req, res) => {

    try {
```

```
const updated = await Student.findOneAndUpdate(
  { id: Number(req.params.id) },
  req.body,
  { new: true }
);
res.json(updated);
}
catch (err) {
  res.status(500).send("Error");
}
});

// DELETE operation
app.delete("/data/:id", async (req, res) => {
  try {
    const deleted = await Student.findOneAndDelete({
      id: Number(req.params.id),
    });
    res.json(deleted);
  }
  catch (err) {
    res.status(500).send("Error");
  }
});
```



```
});  
  
// Start server  
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

Step 8 : Run the Application

node app.js

You should see:

Server running on port 3000

MongoDB connected!

Step 9 : Open Browser and Test API Routes

http://localhost:3000/data

You will see student data in JSON format.

Output

Paste Output Screenshot Here

Program 4 : Create a library of reusable React components. Build components like buttons, cards, and modals and use them in a sample React application.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create a React Application

Open terminal and run the following command:

```
npx create-react-app react-components
```

Step 3 : Open Project Folder

```
cd react-components
```

Step 4 : Create Components Folder

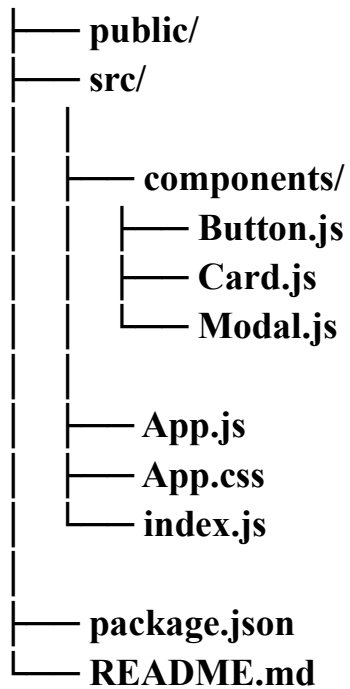
Inside the src folder create a new folder named:

```
components
```

Step 5 : Create Required Files

Project Structure

```
react-components/  
|  
├── node_modules/
```



Step 6 : Write the Following Code in App.js

```
import React, { useState } from "react";

import "./App.css";

import Button from "./components/Button";
import Card from "./components/Card";
import Modal from "./components/Modal";

function App() {

  const [showModal, setShowModal] = useState(false);

  return (

    <div className="app">

      <Button
        text="Open Modal"
        onClick={() => setShowModal(true)}
      />

      <Card>
```

```
    <h3>React Card</h3>

    <p>This is a reusable card component.</p>

  </Card>

  <Modal
    show={showModal}
    onClose={() => setShowModal(false)}
  >

    <h2>Modal Window</h2>

    <p>This is a reusable modal component.</p>

  </Modal>

</div>

);

}

export default App;
```

Step 8 : Write the Following Code in Card.js

```
import React from "react";

// Reusable card component
function Card({ children }) {

  return (

    <div className="card">

      {children}

    </div>

  );
}
```

```
);  
  
}  
  
export default Card;
```

Step 9 : Write the Following Code in Modal.js

```
import React from "react";  
  
// Reusable modal component  
function Modal({ show, onClose, children }) {  
  
  if (!show) {  
  
    return null;  
  
  }  
  
  return (  
  
    <div className="modal-overlay">  
  
      <div className="modal">  
  
        <button  
          className="close-button"  
          onClick={onClose}  
        >  
          X  
        </button>  
  
        {children}  
  
      </div>  
  
    </div>  
  
  );  
  
}
```

```
export default Modal;
```

Step 10 : Write the Following Code in App.css

```
.app {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  height: 100vh;  
}  
  
.button {  
  padding: 10px 20px;  
  margin-bottom: 20px;  
}  
  
.card {  
  border: 1px solid #ccc;  
  padding: 20px;  
  margin-bottom: 20px;  
  width: 300px;  
  text-align: center;  
}
```

```
.modal-overlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background: rgba(0,0,0,0.5);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}  
  
.modal {  
  background: white;  
  padding: 20px;  
  width: 400px;  
  border-radius: 8px;  
  position: relative;  
}  
  
.close-button {  
  position: absolute;
```

```
top: 10px;  
right: 10px;  
cursor: pointer;  
background: none;  
border: none;  
font-size: 18px;  
}
```

Step 11 : Run the React Application

npm start

Step 12 : Open Browser

<http://localhost:3000>

You will see reusable React components displayed in the browser.

Output



Program 5 : Write a program to create a simple Calculator Application using React JS.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create a React Application

Open terminal and run the following command:

```
npx create-react-app react-calculator
```

Step 3 : Open Project Folder

```
cd react-calculator
```

Step 4 : Create Required Files

Project Structure

react-calculator/

```
|
|— node_modules/
|— public/
|— src/
|   |— App.js
|   |— App.css
|   |— index.js
|— package.json
|— README.md
```

Step 5 : Write the Following Code in App.js

```
import React, { useState } from 'react';

import './App.css';

const App = () => {

  const [question, setQuestion] = useState("");

  const [answer, setAnswer] = useState("");

  // Handle button click
  const handleClick = (value) => {

    if (value === '=') {

      try {

        setAnswer(eval(question) || "Math Error");

        setQuestion("");

      }

      catch {

        setAnswer("Math Error");

      }

    }

    else if (value === 'C') {

      setQuestion("");

      setAnswer("");

    }

  }

}
```

```
    else if (value === 'CE') {

        setQuestion(question.slice(0, -1));

    }

    else {

        setQuestion(question + value);

    }

};

// Calculator buttons
const buttons = [

    '1','2','3','4',
    '5','6','7','8',
    '9','-','!','0',
    '=', '*', 'C', 'CE', '/'

];

return (

    <div className="calculator">

        <h2>React Calculator</h2>

        <div className="screen">

            <input
                type="text"
                value={question}
                readOnly
            />

            <input
                type="text"
                value={answer}
```

```
        readOnly
      />

    </div>

    <div className="buttons">

      {buttons.map((btn) => (

        <button
          key={btn}
          onClick={() => handleClick(btn)}
        >

          {btn}

        </button>

      )))}

    </div>

  </div>

);

};

export default App;
```

Step 6 : Write the Following Code in App.css

```
body {

  display: flex;

  justify-content: center;

  align-items: center;

  height: 100vh;
```

```
background-color: #f1f5f9;

font-family: Arial, Helvetica, sans-serif;
}

/* Calculator container */
.calculator {

width: 300px;

background: #1e293b;

padding: 30px;

border-radius: 10px;

box-shadow: 0 6px 12px rgba(0,0,0,0.2);

text-align: center;

color: #ffffff;
}

h2 {

margin-bottom: 10px;
}

/* Display screen */
.screen {

display: flex;

flex-direction: column;

margin-bottom: 10px;
```

```
}

.screen input {

  width: 100%;

  padding: 10px;

  font-size: 1.2em;

  text-align: right;

  margin-bottom: 5px;

  border: 1px solid #cbd5f5;

  border-radius: 5px;

  background: #f8fafc;

  color: #111827;

}

/* Buttons grid */
.buttons {

  display: grid;

  grid-template-columns: repeat(4,1fr);

  gap: 5px;

}

/* Calculator buttons */
button {

  padding: 15px;
```

```
font-size: 1.2em;

border: none;

background: #e2e8f0;

cursor: pointer;

border-radius: 5px;

transition: background 0.2s;
}

/* Hover effect */
button:hover {

    background: #cbd5f5;
}

/* Click effect */
button:active {

    background: #94a3b8;
}
```

Step 7 : Run the React Application

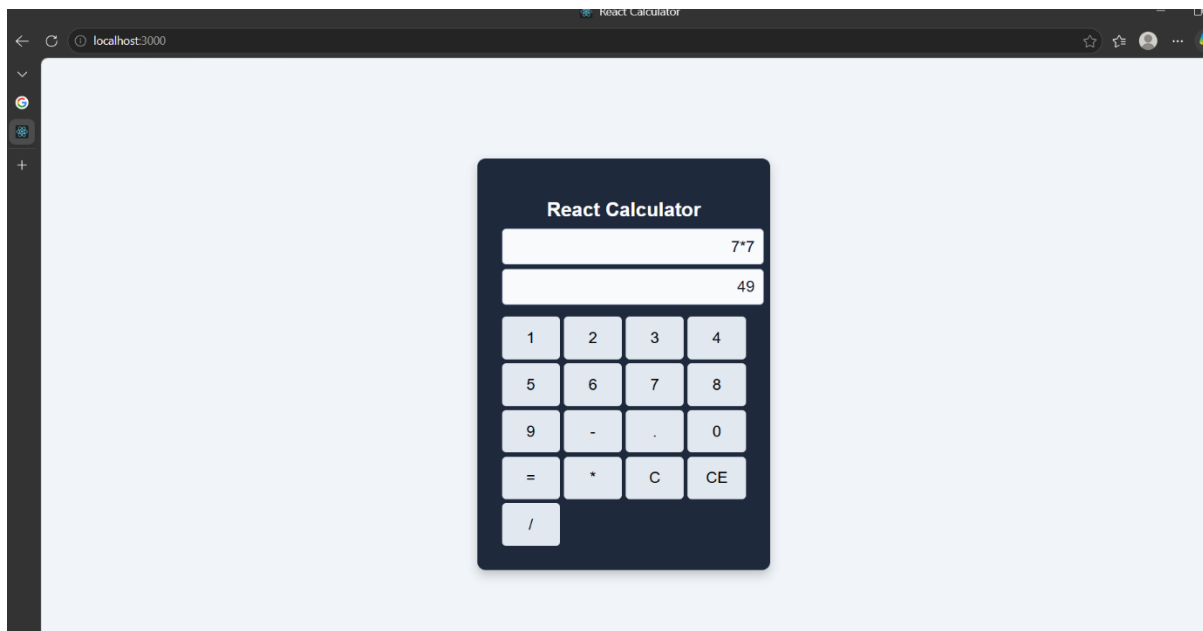
npm start

Step 8 : Open Browser

http://localhost:3000

You will see the Calculator Application in the browser.

Output



Program 6 : Create a Simple Login form using React JS.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create a React Application

Open terminal and run the following command:

```
npx create-react-app react-login
```

Step 3 : Open Project Folder

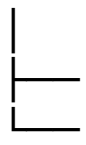
```
cd react-login
```

Step 4 : Create Required Files

Project Structure

react-login/

```
|  
|— node_modules/  
|— public/  
|— src/  
|   |  
|   |— Login.js  
|   |— prog6.css  
|   |— index.js
```



package.json
README.md

Step 5 : Write the Following Code in Login.js

```
import React, { useState } from "react";

import "./prog6.css";

// Login component
function Login() {

  const [username, setUsername] = useState("");

  const [password, setPassword] = useState("");

  // Handle login validation
  const handleLogin = () => {

    if (username === "admin" && password === "123") {

      alert("Login successful");

    }

    else {

      alert("Invalid credentials");

    }

  };

  return (

    <div className="container">
```

```
<h2>Login Form</h2>

<input
  type="text"
  placeholder="Username"
  onChange={(e) => setUsername(e.target.value)}
/>

<br /><br />

<input
  type="password"
  placeholder="Password"
  onChange={(e) => setPassword(e.target.value)}
/>

<br /><br />

<button onClick={handleLogin}>

  Login

</button>

</div>

);

}

export default Login;
```

Step 6 : Write the Following Code in prog6.css

```
body {

  display: flex;

  justify-content: center;

  align-items: center;
```

```
height: 100vh;

background-color: #f1f5f9;

font-family: Arial, Helvetica, sans-serif;
}

/* Login container */
.container {

background: white;

padding: 30px;

border-radius: 10px;

box-shadow: 0 6px 12px rgba(0,0,0,0.2);

text-align: center;

width: 300px;

}

h2 {

margin-bottom: 20px;

}

/* Input fields */
input {

width: 90%;

padding: 10px;

border: 1px solid #ccc;
```

```
border-radius: 5px;

font-size: 16px;

}

/* Login button */
button {

width: 100%;

padding: 10px;

border: none;

border-radius: 5px;

background-color: #2563eb;

color: white;

font-size: 16px;

cursor: pointer;

}

/* Hover effect */
button:hover {

background-color: #1d4ed8;

}
```

Step 7 : Update index.js File

```
import React from "react";
import ReactDOM from "react-dom/client";

import Login from "../Login";
```

```
const root = ReactDOM.createRoot(  
  document.getElementById("root")  
);  
  
root.render(  
  <React.StrictMode>  
    <Login />  
  </React.StrictMode>  
);
```

Step 8 : Run the React Application

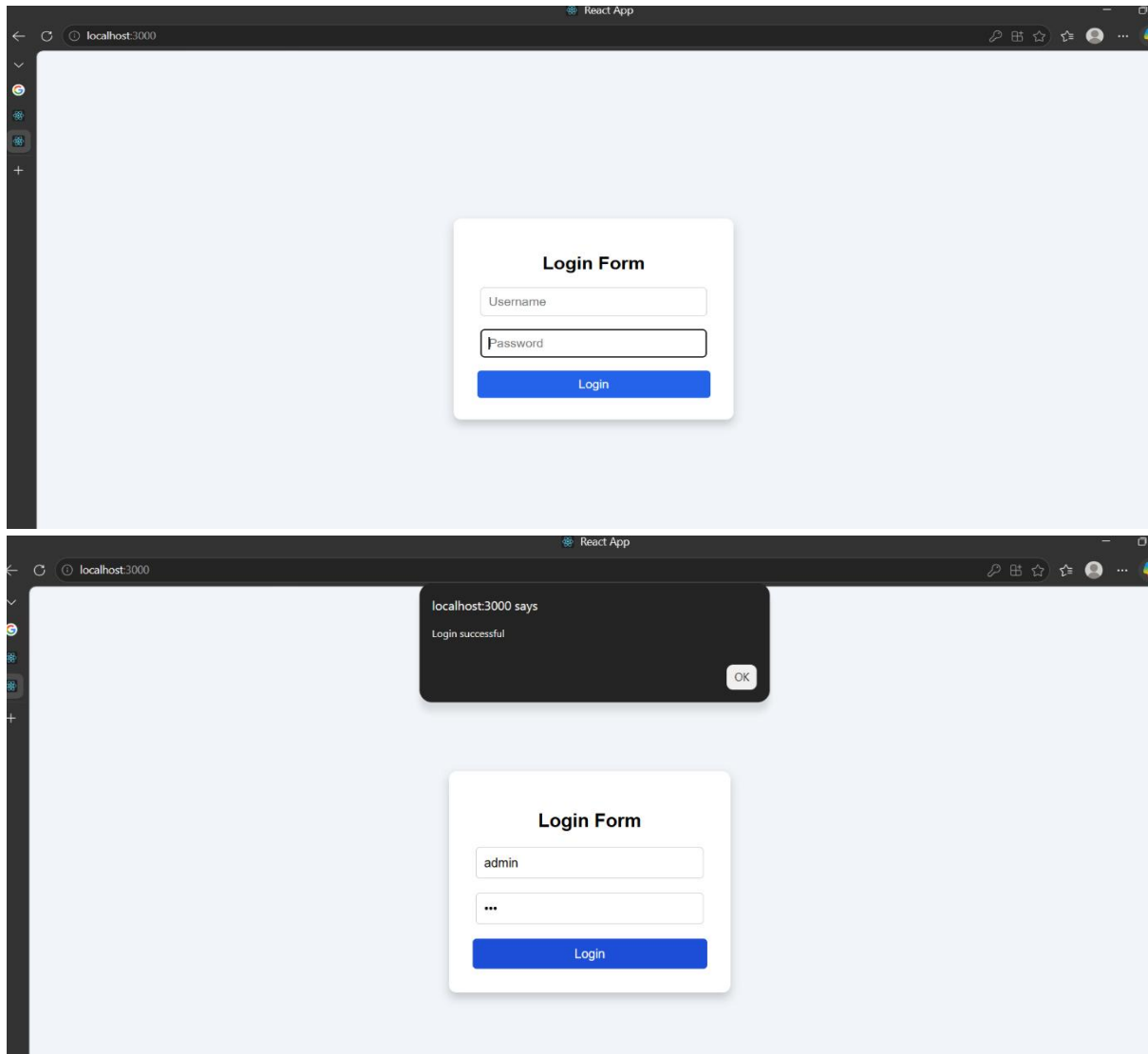
npm start

Step 9 : Open Browser

<http://localhost:3000>

You will see the Login Form in the browser.

Output



Program 7 : Build a real-time chat application using React for the front end and Node.js with Socket.IO for the back end. Enable users to send and receive messages in real-time.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Create Backend Folder

Create a folder named server and open it in VS Code.

Step 3 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 4 : Initialize Node.js Project

```
npm init -y
```

This creates a default package.json file.

Step 5 : Install Required Packages

```
npm install express socket.io cors
```

This installs Express, Socket.IO, and CORS packages.

Step 6 : Create server.js File

Project Structure

server/

- |
- |— **node_modules/**
- |— **package.json**
- |— **server.js**

Step 7 : Write the Following Code in server.js

```
const express = require("express");

const http = require("http");

const cors = require("cors");

const { Server } = require("socket.io");

// Create Express application
const app = express();

app.use(cors());

// Create HTTP server
const server = http.createServer(app);

// Configure Socket.IO
const io = new Server(server, {

  cors: {

    origin: "http://localhost:3000",

    methods: ["GET", "POST"]

  }

});
```

```
// Socket connection
io.on("connection", (socket) => {

  console.log("User connected:", socket.id);

  // Receive and broadcast message
  socket.on("send_message", (data) => {

    io.emit("receive_message", data);

  });

  // Disconnect event
  socket.on("disconnect", () => {

    console.log("User disconnected:", socket.id);

  });

});

// Start server
server.listen(5000, () => {

  console.log("Server running on port 5000");

});
```

Step 8 : Run Backend Server

node server.js

You should see:

Server running on port 5000

Step 9 : Create Frontend React Application

Open a new terminal and run:

```
npx create-react-app chat-app
```

Step 10 : Open React Project Folder

```
cd chat-app
```

Step 11 : Install Socket.IO Client

```
npm install socket.io-client
```

Step 12 : Create Required Files

Project Structure

```
chat-app/
├──
├── node_modules/
├── public/
├── src/
│   ├── App.js
│   └── index.js
├── package.json
└── README.md
```

Step 13 : Write the Following Code in App.js

```
import React, { useEffect, useState } from "react";

import io from "socket.io-client";

// Connect frontend to backend
```

```
const socket = io("http://localhost:5000");

function App() {

  const [username, setUsername] = useState("");

  const [message, setMessage] = useState("");

  const [chat, setChat] = useState([]);

  const [joined, setJoined] = useState(false);

  // Join chat room
  const joinChat = () => {

    if (username !== "") {

      setJoined(true);

    }

  };

  // Send message
  const sendMessage = () => {

    if (message !== "") {

      socket.emit("send_message", {

        user: username,

        text: message

      });

      setMessage("");

    }

  }

}
```

```
};

// Receive messages
useEffect(() => {

  socket.on("receive_message", (data) => {

    setChat((prev) => [...prev, data]);

  });

}, []);

return (

  <div style={{ padding: "20px" }}>

    {!joined ? (

      <div>

        <h2>Enter Username</h2>

        <input
          type="text"
          placeholder="Your name"
          onChange={(e) => setUsername(e.target.value)}
        />

        <button onClick={joinChat}>

          Join Chat

        </button>

      </div>

    ) : (

      <div>
```

```
<h2>Welcome {username}</h2>

<div style={{
  border: "1px solid black",
  height: "300px",
  padding: "10px",
  overflowY: "scroll"
}}>

  {chat.map((msg, index) => (
    <p key={index}>
      <b>{msg.user}</b> {msg.text}
    </p>
  ))}

</div>

<input
  type="text"
  value={message}
  onChange={(e) => setMessage(e.target.value)}
/>

<button onClick={sendMessage}>

  Send

</button>

</>

  )}

</div>
```

```
);  
  
}  
  
export default App;
```

Step 14 : Run the React Application

npm start

Step 15 : Open Browser

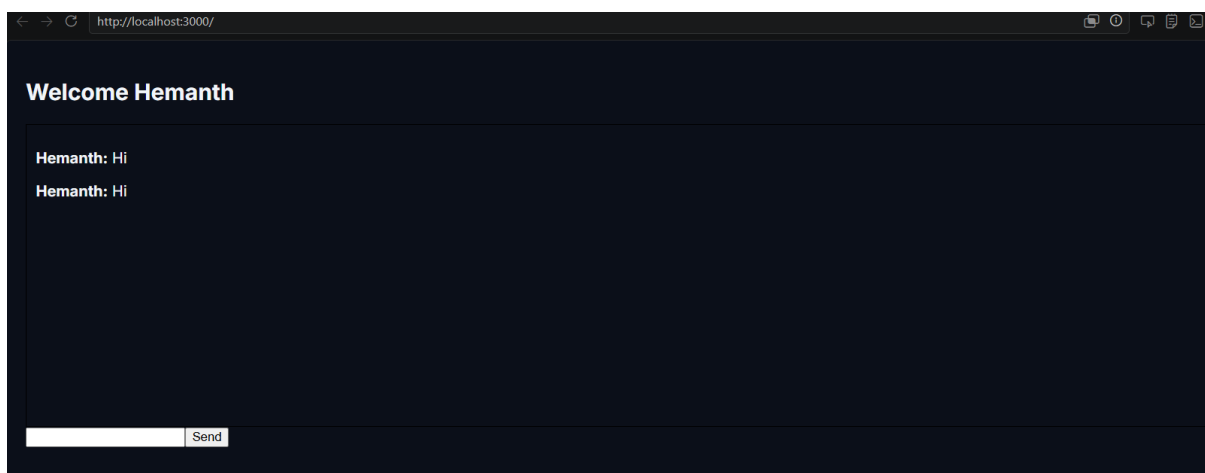
http://localhost:3000

You will see the Real-Time Chat Application in the browser.

Output



Enter Username



Program 8 : Develop a Django app that displays current date and time in server.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 3 : Install Django

```
python -m pip install Django
```

This installs the Django framework.

Step 4 : Create Django Project

```
python -m django startproject datetime_project
```

Step 5 : Open Project Folder

```
cd datetime_project
```

Step 6 : Create Django App

```
python manage.py startapp timeapp
```

Step 7 : Register App in settings.py

Open:

datetime_project/settings.py

Add the following line inside INSTALLED_APPS

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'timeapp',  
]
```

Step 8 : Create View in views.py

Open:

timeapp/views.py

Write the following code:

```
from django.http import HttpResponse  
  
from datetime import datetime  
  
# Display current server date and time  
def current_datetime(request):  
    now = datetime.now()  
  
    html = f"<h1>Current Date and Time (Server): {now}</h1>"
```

```
return HttpResponse(html)
```

Step 9 : Configure URLs

Open:

`datetime_project/urls.py`

Write the following code:

```
from django.contrib import admin

from django.urls import path

from timeapp import views

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", views.current_datetime),

]
```

Step 10 : Project Structure

datetime_project/

```
|
|— datetime_project/
|   |— settings.py
|   |— urls.py
|   |— wsgi.py
|
|— timeapp/
|   |— views.py
|   |— admin.py
|
|— manage.py
|— db.sqlite3
```

Step 11 : Run the Django Server

```
python manage.py runserver
```

You should see:

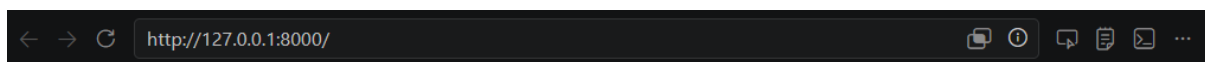
Starting development server at <http://127.0.0.1:8000/>

Step 12 : Open Browser

<http://127.0.0.1:8000/>

You will see the current date and time displayed in the browser.

Output



**Current Date and Time (Server): 2026-05-21
21:57:45.737814**

Program 9 : Develop a simple Django app that displays an unordered list of fruits and ordered list of selected students for an event.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 3 : Create Django Project

```
python -m django startproject list_project
```

Step 4 : Open Project Folder

```
cd list_project
```

Step 5 : Create Django App

```
python manage.py startapp listapp
```

Step 6 : Register App in settings.py

Open:

```
list_project/settings.py
```

Add the following line inside `INSTALLED_APPS`

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'listapp',  
]
```

Step 7 : Create View in views.py

Open:

listapp/views.py

Write the following code:

```
from django.shortcuts import render  
  
# Display fruits and students list  
def display_lists(request):  
    fruits = ['Apple', 'Banana', 'Mango', 'Orange']  
    students = ['Rahul', 'Anita', 'Kiran', 'Sneha']  
    return render(request, 'listapp/index.html', {  
        'fruits': fruits,  
        'students': students  
    })
```

Step 8 : Create Templates Folder

Create the following folder structure manually:

```
listapp/  
├── templates/  
│   └── listapp/  
│       └── index.html  
├── views.py  
└── admin.py
```

Step 9 : Write the Following Code in index.html

Open:

templates/listapp/index.html

Write the following code:

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Lists Example</title>  
</head>  
  
<body>  
    <h2>Fruits (Unordered List)</h2>  
    <ul>  
        {% for fruit in fruits %}  
            <li>{{ fruit }}</li>  
        {% endfor %}
```

```
</ul>

<h2>Selected Students (Ordered List)</h2>

<ol>

    {% for student in students %}

        <li>{{ student }}</li>

    {% endfor %}

</ol>

</body>

</html>
```

Step 10 : Configure URLs

Open:

list_project/urls.py

Write the following code:

```
from django.contrib import admin

from django.urls import path

from listapp import views

urlpatterns = [

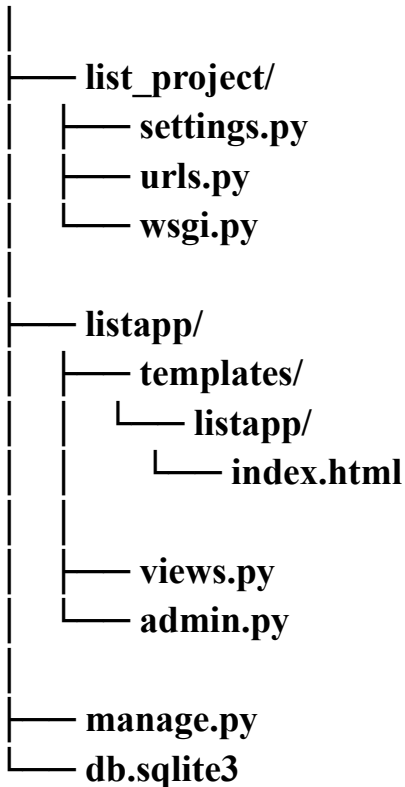
    path('admin/', admin.site.urls),

    path("", views.display_lists),

]
```

Step 11 : Project Structure

list_project/



Step 12 : Run the Django Server

```
python manage.py runserver
```

You should see:

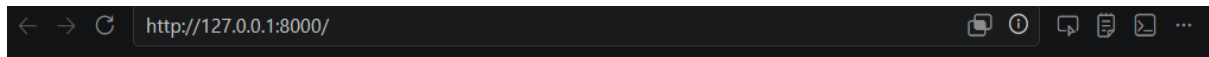
Starting development server at <http://127.0.0.1:8000/>

Step 13 : Open Browser

<http://127.0.0.1:8000/>

You will see the unordered list of fruits and ordered list of students in the browser.

Output



Fruits (Unordered List)

- Apple
- Banana
- Mango
- Orange

Selected Students (Ordered List)

1. Rahul
2. Anita
3. Kiran
4. Sneha

Program 10 : Develop a Django app that performs student registration on a course. It should also display a list of students registered for any selected course. Create students and course as models with enrolment as many to many fields.

Execution Procedure

Step 1 : Open VS Code

Open Visual Studio Code on your system.

Step 2 : Open Terminal in VS Code

Open the integrated terminal using:

Terminal → New Terminal

Step 3 : Create Django Project

```
python -m django startproject myproject
```

Step 4 : Open Project Folder

```
cd myproject
```

Step 5 : Create Django App

```
python manage.py startapp myapp
```

Step 6 : Register App in settings.py

Open:

```
myproject/settings.py
```

Add the following line inside `INSTALLED_APPS`

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'myapp',  
]
```

Step 7 : Create Models in `models.py`

Open:

`myapp/models.py`

Write the following code:

```
from django.db import models  
  
# Course model  
class Course(models.Model):  
    name = models.CharField(max_length=50)  
    def __str__(self):  
        return self.name  
  
# Student model  
class Student(models.Model):
```

```
name = models.CharField(max_length=50)

courses = models.ManyToManyField(Course)

def __str__(self):

    return self.name
```

Step 8 : Apply Database Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

These commands create database tables.

Step 9 : Create View in views.py

Open:

```
myapp/views.py
```

Write the following code:

```
from django.shortcuts import render

from .models import Student, Course

# Display registered students
def home(request):

    courses = Course.objects.all()

    students = None

    if request.GET.get('course'):

        course_id = request.GET.get('course')
```

```
students = Course.objects.get(
    id=course_id
).student_set.all()

return render(request, 'home.html', {
    'courses': courses,
    'students': students
})
```

Step 10 : Configure App URLs

Create a new file named:

myapp/urls.py

Write the following code:

```
from django.urls import path

from .views import home

urlpatterns = [
    path("", home),
]
```

Step 11 : Configure Main URLs

Open:

myproject/urls.py

Write the following code:

```
from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", include('myapp.urls')),

]
```

Step 12 : Create Templates Folder

Create the following folder structure manually:

```
myapp/
├── templates/
│   └── home.html
├── models.py
├── views.py
└── urls.py
```

Step 13 : Write the Following Code in home.html

Open:

templates/home.html

Write the following code:

```
<h2>Select Course</h2>

<form method="GET">

    <select name="course">

        {% for c in courses %}
```

```
<option value="{{ c.id }}">
    {{ c.name }}
</option>
{% endfor %}
</select>

<button type="submit">
    Show Students
</button>
</form>

{% if students %}
    <h3>Students:</h3>
    <ul>
        {% for s in students %}
            <li>{{ s.name }}</li>
        {% endfor %}
    </ul>
{% endif %}
```

Step 14 : Project Structure

myproject/

|

├── myproject/

| ├── settings.py

```
| | | urls.py
| | | wsgi.py
| | |
| | | myapp/
| | | | templates/
| | | | | home.html
| | | |
| | | | models.py
| | | | views.py
| | | | urls.py
| | |
| | | manage.py
| | | db.sqlite3
```

Step 15 : Run the Django Server

```
python manage.py runserver
```

You should see:

Starting development server at <http://127.0.0.1:8000/>

Step 16 : Create Superuser

```
python manage.py createsuperuser
```

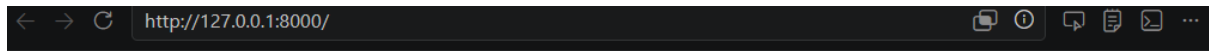
This creates an admin login account.

Step 17 : Open Browser

<http://127.0.0.1:8000/>

You will see the Course Registration Application in the browser.

Output



Select Course

Python



Select Course

Python

Students:

- Abhishek
- Chetan
- Rahul
- Kiran

